

W12D01 — Multi-stage Docker Build

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 25 Mei 2026

1. Analogi: Bengkel vs Meja Jadi

Bayangkan kamu memesan meja kayu custom dari seorang pengrajin. Pengrajin itu butuh gergaji, amplas, bor, cat primer, mesin poles, dan puluhan alat lainnya untuk *membuat* mejanya. Tapi ketika meja itu tiba di rumahmu — kamu tidak butuh semua alat itu ikut dikirim. Kamu hanya butuh **mejanya**.

Single-stage Docker build itu seperti pengrajin yang mengirim meja **beserta seluruh bengkelnya** ke rumahmu. 600MB itu adalah meja *plus* gergaji, bor, dan semua sampah serbuk kayu yang tersisa.

Multi-stage build memisahkan dua fase dengan tegas:

- **Builder stage** — bengkel pengrajin. Boleh kotor, boleh besar, tidak ada yang peduli.
- **Production stage** — hanya mejanya. Bersih, ringan, siap pakai.

2. Definisi Teknis

Multi-stage build adalah fitur Dockerfile yang memungkinkan kamu mendefinisikan **lebih dari satu instruksi FROM** dalam satu file. Setiap FROM membuat stage baru yang sepenuhnya independen — layer dari stage sebelumnya tidak otomatis ikut ke stage berikutnya.

Kamu bisa menyalin artifact dari stage sebelumnya menggunakan **COPY --from=<stage_name>**. Stage yang tidak di-copy ke final image tidak ikut masuk ke image akhir — mereka hanya exist selama proses build. Ini yang menjadi mekanisme isolasi: setiap FROM adalah container virtual baru yang bersih.

3. Perbandingan: Single-stage vs Multi-stage

Aspek	Single-stage	Multi-stage
Build tools di final image	Ikut masuk	Tidak ikut
pip cache di final image	Ikut masuk	Tidak ikut
Ukuran image (web app)	~600MB+	~180–250MB
Ukuran image (ML stack)	~700MB	~677MB*
Attack surface	Lebih besar	Lebih kecil
gcc di production	Ada	Tidak ada
Cocok production	Tidak ideal	Ya

* Untuk ML stack, penurunan ukuran kecil karena scikit-learn modern di PyPI sudah shipping sebagai **pre-compiled wheels** — gcc di builder tidak pernah benar-benar dipakai compile. Gain utama multi-stage untuk ML bukan ukuran, tapi **keamanan**: gcc tidak ada di production artinya attacker tidak bisa compile exploit di dalam container.

4. Implementasi Dockerfile Multi-stage

Stage 1 — Builder

```
FROM python:3.11-slim AS builder

# Build dependencies – hanya untuk compile C extensions
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /build

# Requirements dulu – manfaatkan layer caching
COPY requirements.txt .

# --prefix=/install: semua package ke folder khusus
# Ini yang akan di-copy ke production stage
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
```

Stage 2 — Production

```
FROM python:3.11-slim AS production

# Python tidak tulis .pyc, output tidak di-buffer
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PYTHONPATH=/app

WORKDIR /app

# HANYA hasil install dari builder – gcc tidak ikut
COPY --from=builder /install /usr/local

# Source code di layer paling bawah – rebuild cepat saat kode berubah
COPY app/ ./app/

# Non-root user – security best practice
RUN adduser --disabled-password --gecos '' appuser && \
    chown -R appuser:appuser /app
```

USER appuser

EXPOSE 8000

Exec form: signal langsung ke uvicorn (bukan via shell)

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Penjelasan Baris Kritis

FROM ... AS builder	Keyword AS memberi nama stage. Nama ini yang direferensikan di COPY --from=.
pip install --prefix=/install	Flag --prefix mengubah lokasi instalasi ke /install. Ini memudahkan kita copy hanya folder itu ke stage berikutnya — bukan seluruh /usr/local.
FROM python:3.11-slim AS production	Fresh start total. Layer dari stage builder tidak ada di sini. Image mulai bersih dari base Python slim.
COPY --from=builder /install /usr/local	Inti multi-stage. Copy hasil compile dari builder ke /usr/local di production — lokasi default Python mencari packages. gcc tidak ikut.
ENV PYTHONDONTWRITEBYTECODE=1	Python tidak buat file .pyc di container. Hemat disk, image lebih bersih.
ENV PYTHONUNBUFFERED=1	Output Python langsung ke stdout tanpa buffer. Kritis untuk Docker log collection — tanpa ini log bisa delay atau hilang.
USER appuser	Jangan jalankan app sebagai root di production. Kalau ada vulnerability di app, attacker tidak otomatis dapat root access ke container.
CMD exec form	Array syntax: signal (SIGTERM) langsung ke uvicorn sebagai PID 1. Shell form membungkus dalam /bin/sh -c sehingga signal tidak diteruskan dengan benar.

5. Rekap Debug Session — 4 Error Berturut-turut

Hari ini terjadi 4 error sebelum app berhasil jalan. Ini adalah pelajaran paling berharga dari sesi ini — setiap error berasal dari layer yang berbeda dan membutuhkan diagnosis yang berbeda.

Error 1 — Missing Environment Variable

Command	<code>docker run -p 8000:8000 risk-api:multistage</code>
Error	<code>pydantic_core.ValidationError: api_key — Field required</code>
Root Cause	<code>config.py</code> mendefinisikan <code>api_key</code> dengan <code>Field(...)</code> yang berarti <code>required</code> tanpa default. <code>Pydantic-settings</code> mencari env var <code>API_KEY</code> , tidak ketemu, crash di baris <code>settings = Settings()</code> sebelum app sempat jalan. Ini fail-fast by design dari W07.
Fix	Tambahkan <code>--env-file .env.dev</code> ke command <code>docker run</code> .

Error 2 — Model File Tidak Ditemukan

Command	<code>docker run -p 8000:8000 --env-file .env.dev risk-api:multistage</code>
Error	<code>FileNotFoundError: /app/model/pipeline_pipe_v1_20260415.joblib</code>
Root Cause	<code>.dockerignore</code> mengecualikan file <code>.joblib</code> — model tidak ikut di-copy ke image saat build. Ini benar dan disengaja. Tapi saat run tanpa volume mount, container tidak punya akses ke model.
Fix	Mount folder model via <code>-v</code> flag: <code>-v "\${PWD}/app/model:/app/app/model"</code>

Error 3 — Path Mismatch (paling subtle)

Command	<code>docker run</code> dengan volume mount <code>/app/app/model</code>
Error	<code>FileNotFoundError: /app/model/...</code> (message sama dengan Error 2, penyebab berbeda)
Root Cause	<code>.env.dev</code> berisi <code>MODEL_PATH=/app/model/...</code> yang override kalkulasi <code>BASE_DIR</code> di <code>config.py</code> . <code>BASE_DIR</code> resolve ke <code>/app</code> di container, sehingga <code>config.py</code> expect <code>/app/app/model/</code> . Tapi env var override ke <code>/app/model/</code> . Miss by one directory level.
Fix	Sesuaikan volume mount target ke <code>/app/model</code> : <code>-v "\${PWD}/app/model:/app/model"</code>

Error 4 — Redis Hostname Tidak Bisa Di-resolve

Command	<code>docker run standalone</code> (berhasil load model)
Error	<code>socket.gaierror: [Errno -2] Name or service not known — redis:6379</code>
Root Cause	<code>redis_host='redis'</code> di config adalah Docker internal hostname, hanya bisa di-resolve kalau ada service bernama 'redis' di Docker network yang sama. <code>docker run standalone</code> masuk ke default bridge network tanpa service redis.

Fix

Gunakan Docker Compose yang mendefinisikan kedua service dalam satu network. Fix `docker-compose.dev.yml`: tambahkan `image: risk-api:multistage` di service `api` dan `image: redis:7-alpine` di service `redis`.

6. Pattern Penting yang Harus Diingat

POLA DEBUG DOCKER

Dua error bisa punya message yang identik tapi root cause yang berbeda. Error 2 dan Error 3 sama-sama `FileNotFoundError` untuk path yang sama — tapi satu karena volume tidak di-mount, satunya karena env var override path. Selalu eliminasi satu layer per satu: `environment` → `filesystem` → `path resolution` → `networking`.

ML STACK vs WEB APP

Untuk ML stack, gain multi-stage bukan di ukuran (scikit-learn pre-compiled wheels sudah besar). Gain utamanya di keamanan: `gcc` tidak ada di production image berarti attacker tidak bisa compile exploit. Jangan ukur keberhasilan multi-stage hanya dari MB.

MODEL FILE BEST PRACTICE

Model file tidak seharusnya di-bake ke Docker image. Mount via volume (`-v`) untuk development, atau gunakan object storage (S3, GCS) untuk production. Image tetap ringan dan reusable tanpa terikat ke versi model tertentu.

DOCKER NETWORKING

`redis_host='redis'` hanya bisa di-resolve di Docker network yang sama. Selalu gunakan Docker Compose untuk multi-service stack — jangan `docker run` standalone kalau app kamu butuh service lain (Redis, PostgreSQL, dll).

7. Cara Verifikasi Build

```
# Build production stage

docker build -t risk-api:multistage .

# Bandingkan ukuran

docker images | Where-Object { $_ -match "risk-api" }

# Jalankan via Compose (dengan Redis)

docker compose -f docker-compose.dev.yml up

# Test health endpoint

curl http://localhost:8000/health
```

```
# Inspect isi image
```

```
docker run --rm --entrypoint sh risk-api:multistage -c "du -sh /usr/local/lib/python3.11  
/site-packages"
```